Titanic Survival Prediction

Hasti Aksoy

September 26, 2025

1 Introduction

On April 15, 1912, the RMS Titanic sank after colliding with an iceberg during its maiden voyage from Southampton to New York. Of the 2,224 people on board, 1,502 lost their lives. The tragedy is one of the most studied maritime disasters in history and has become a canonical dataset for binary classification tasks in machine learning, thanks to the Kaggle competition *"Titanic: Machine Learning from Disaster."*

The goal of this project is to predict whether a passenger survived the Titanic disaster, given a set of socio-demographic and travel-related features. The training dataset contains 891 labeled passengers. Several models were developed and compared, including Logistic Regression, Random Forest, Gradient Boosting, Support Vector Machine (SVM), and k-Nearest Neighbors (KNN). The pipeline was designed to ensure reproducibility, avoid data leakage, and address issues such as missing values and class imbalance.

2 Dataset and Data Cleaning

The training dataset consists of 12 columns. These include the target variable (`Survived`), the identifier (`PassengerId`), and 10 predictive features such as `Pclass`, `Name`, `Sex`, `Age`, `SibSp`, `Parch`, `Ticket`, `Fare`, `Cabin`, and `Embarked`.

2.1 Missing Values

- `Age`: 714 non-null values (177 missing, ≈20%)
- `Cabin`: 204 non-null values (687 missing, ≈77%)
- `Embarked`: 889 non-null values (2 missing)

2.2 Cleaning and Feature Engineering (in `01_cleaning.ipynb`)

- **Parsing and Extraction:** Extracted **Title** from `Name`, **Deck** from `Cabin`, and **TicketPrefix** from `Ticket`.
- **Handling Missing Values:** Imputed `Embarked` with the mode, filled `Fare` with the median per `Pclass`, but left `Age` unimputed here (median imputation deferred to pipeline).
- **Engineered Features:** Created `FamilySize`, `IsAlone`, `HasCabin`, and `TicketGroupSize`.
- **Column Reduction:** Dropped high-cardinality variables (`Name`, `Ticket`, `Cabin`).
- **Reordering:** Reordered columns for readability, keeping `Survived` first and `PassengerId` last.

Final cleaned dataset columns: `Survived, Pclass, Sex, Age, SibSp, Parch, FamilySize, IsAlone, Fare, Embarked, Deck, HasCabin, TicketGroupSize, TicketPrefix, Title, PassengerId`

3 Exploratory Data Analysis (EDA)

3.1 Target Distribution and Missingness

- Survivors: $\sim$38%
- Non-survivors: $\sim$62%

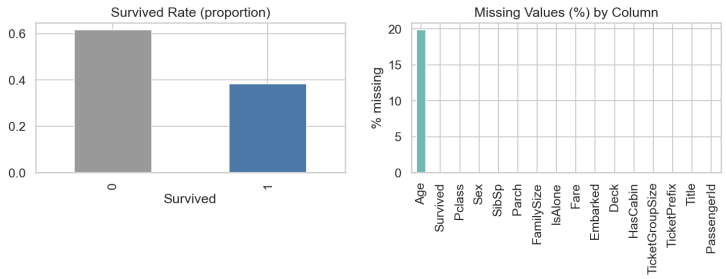

Figure 1: Target distribution and missing values by column.

3.2 Sex and Class

Females had a survival rate of ∼74%, while males had only 19%. First-class passengers survived at 63%, compared to 47% (second class) and 24% (third class).

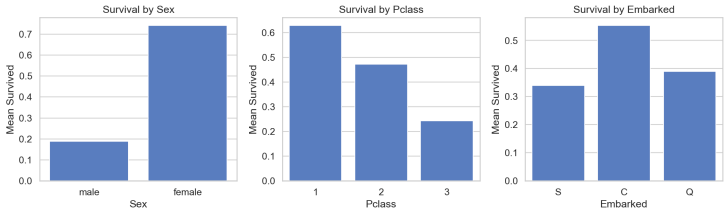

Figure 2: Survival by sex, class, and embarkation port.

3.3 Age and Fare

Children under 10 survived more frequently than adults. Survivors also paid higher fares on average.

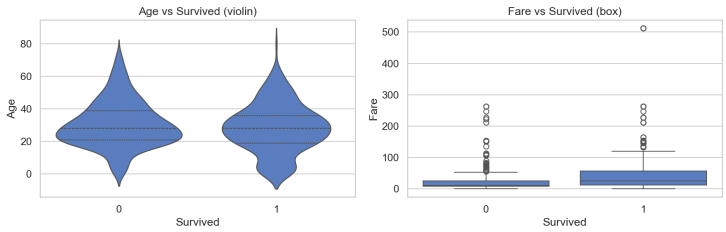

Figure 3: Age and Fare distributions by survival outcome.

3.4 Engineered Features

Traveling alone reduced survival probability (¡30%), while small families (2–4 members) improved it. Having cabin information increased survival.

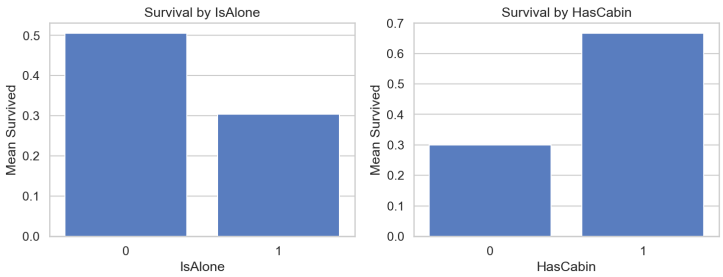

Figure 4: Survival by engineered features (IsAlone, HasCabin).

3.5 Correlation Analysis

Family-related variables were highly correlated; Fare had a positive correlation with survival.

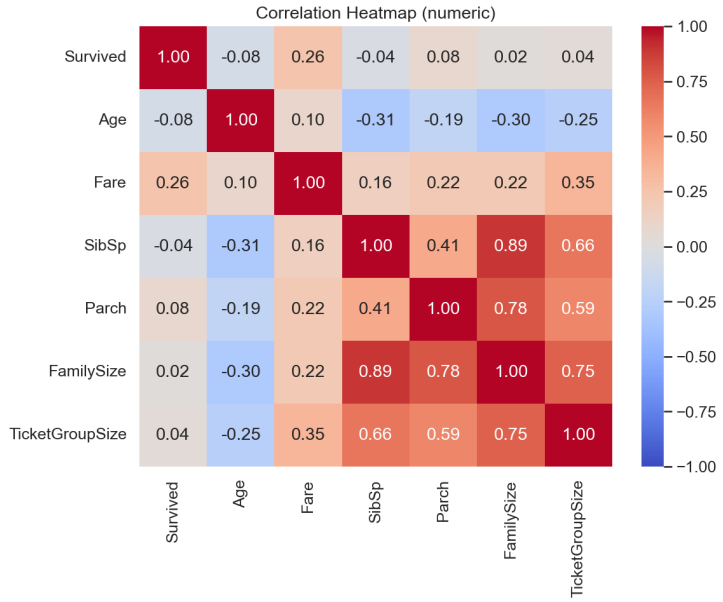

Figure 5: Correlation heatmap of numeric features.

3.6 Feature Importances

Gradient Boosting showed `Title_Mr`, `Fare`, `Pclass_3`, and `Age` as top predictors.

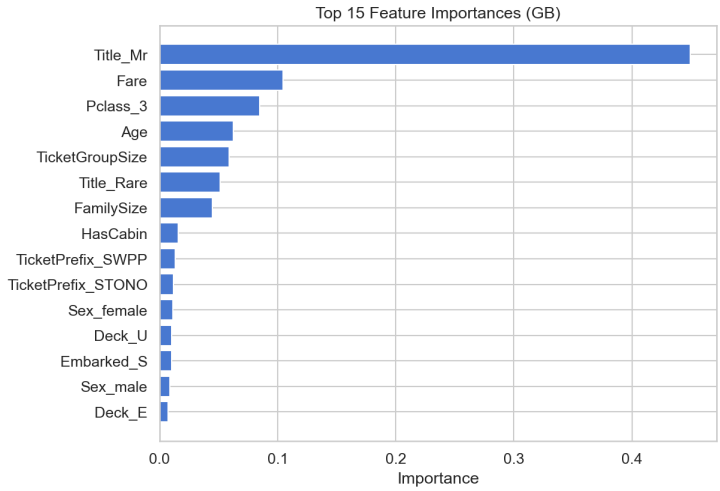

Figure 6: Top 15 feature importances from Gradient Boosting.

4 Preprocessing & Feature Engineering

Implemented in `src/features.py`:

- **Numeric:** Median imputation + StandardScaler.
- **Binary:** Passed through as-is.
- **Categorical:** Most-frequent imputation + One-Hot Encoding.

Age was imputed within the pipeline to avoid leakage. Survived and PassengerId were excluded.

5 Methodology / Modeling Approach

Models defined in `src/models.py`:

- Logistic Regression
- Random Forest (300 trees)
- Gradient Boosting
- Support Vector Machine (SVM, probability=True)
- k-Nearest Neighbors (k=15)

Evaluation used stratified 5-fold CV with metrics: Accuracy, Precision, Recall, F1, and ROC-AUC.

Class imbalance: 38% survivors vs. 62% non-survivors. Class weights were computed:

- Class 0 (non-survivor): 0.81
- Class 1 (survivor): 1.30

These were applied as `sample_weights`.

6 Results

6.1 Cross-Validation Baselines

Model	Accuracy	Precision	Recall	F1
Gradient Boosting	**0.846**	**0.821**	0.748	**0.782**
SVM	0.835	0.807	0.751	0.777
Logistic Regression	0.828	0.789	0.754	0.771
Random Forest	0.814	0.768	0.740	0.753
KNN	0.815	0.798	0.693	0.742

Table 1: Cross-validation performance across baseline models.

6.2 Weighted Gradient Boosting

Applying class weights improved survivor recall:

- Recall(1): 0.71 $\rightarrow$ 0.77
- Accuracy: 0.82 $\rightarrow$ 0.79
- Macro-F1 improved, balancing both classes

6.3 Gradient Boosting Tuning

- Trained with 1200 estimators, learning rate = 0.05
- Staged evaluation found best number = 1141
- Best F1 at threshold 0.5 = 0.766

6.4 Threshold Optimization

- Threshold sweep from 0.05 to 0.95
- Best threshold ≈ 0.47
- Survivor recall further increased while keeping balanced precision/recall

6.5 Final Model Performance (Weighted GB, 1141 trees, tuned threshold)

- Accuracy: 0.816
- Class 0 (non-survivors): Precision 0.86, Recall 0.84, F1 0.85
- Class 1 (survivors): Precision 0.75, Recall 0.78, F1 0.77
- Macro-F1: 0.81

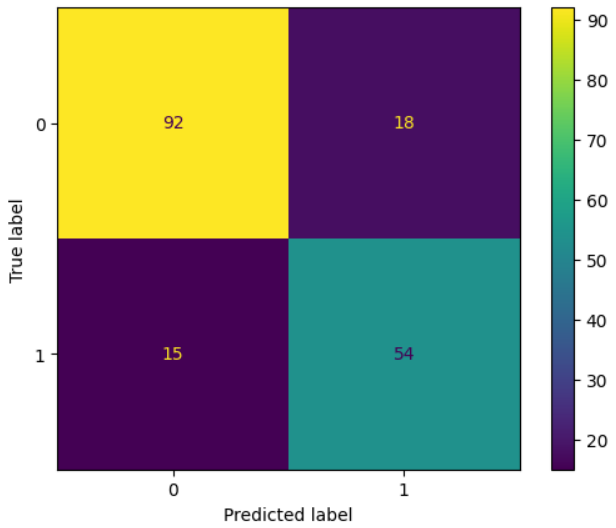

Figure 7: Confusion matrix of final Gradient Boosting model (weighted, tuned).

7 Discussion

- **Why GB won:** Gradient Boosting captured non-linear interactions among sex, fare, age, and family features better than linear or distance-based models.

- **Effect of weights:** Recall for survivors improved by +6pp, reducing false negatives.
- **Effect of tuning:** Optimal trees (1141) avoided overfitting; threshold tuning improved positive-class detection.
- **Trade-offs:** Without weights: higher accuracy but lower survivor recall. With weights: slightly lower accuracy, much better balance.
- **Feature importance:** Top predictors were `Title_Mr`, `Fare`, `Pclass_3`, `Age`, and engineered family/cabin features.
- **Limitations:** Dataset size (n=891), heavy missingness in Cabin, moderate imbalance.
- **Future work:** Hyperparameter optimization (GridSearch, Optuna), advanced gradient boosting (XGBoost, LightGBM), stacking ensembles, iterative imputation for Age.

8 Conclusion

This project shows that classical ML methods can effectively predict Titanic survival. Gradient Boosting, after weighting, tuning, and threshold optimization, achieved the best performance (Accuracy 0.816, Macro-F1 0.81).

Key insights:

- Survival influenced strongly by sex, class, fare, and age.
- Engineered features (`IsAlone`, `HasCabin`, `FamilySize`, `TicketGroupSize`) improved predictive power.
- Ensemble methods outperformed linear and distance-based classifiers.

All code, notebooks, and figures are available on GitHub: `https://github.com/hasti-aksoy`